

My Upgrade: Enhancing Code Structure and Integrating Processes into the Ontology

Master Degree in Computer Science, University of Pisa
Smart Applications
Academic Year 2024/2025

Giacomo Aru - g.aru@studenti.unipi.it - 597700

Abstract

In this report, I will detail all the modifications I have made to the code and the ontology as part of the My Upgrade project. I will explain the rationale behind each change and how it can positively impact the overall project. For the more innovative aspects, I will provide a deeper explanation, highlighting their potential benefits and how they could contribute to the project's success if embraced across the teams.

1 Introduction

The ultimate goal of Topic 1, of which I was the Topic Leader, has always been to create a functional knowledge base (KB) to inform the project about the most relevant concepts. In this “My Upgrade”, I expanded and refined the implementation in two key ways: first, by enhancing several methods and making the interface implementation more effective and functional; and second, by extending the KB with the introduction of Processes and all the necessary satellite concepts.

The introduction of Processes into the Ontology is the central addition of this “My Upgrade”, leading to a structural modification of the KB by broadening its concepts and relationships. This change also required adjustments to the interface and the correction of functions that became inconsistent after the update. Additionally, I improved the overall functionality of the interface, increasing the KB's dynamism by refining functions for creating and deleting entities such as KPIs, processes, and operations.

It is important to note that I did not modify the existing online documentation, as this would have diverted time and resources from the core objectives of the project. Instead, I focused on providing detailed inline documentation for each method in the code itself.

To address each modification with the appropriate level of detail, this report will be structured as follows:

1. **Ontology Expansion: New Entities and Properties:** The first section will provide a detailed explanation of the structural changes made to the Ontology, including the rationale behind each decision.
2. **Code Modifications: Improvements and New Implementations:** The second section will focus on updates to existing code, including both improvements to the current implementations and the introduction of new functionalities developed from scratch.
3. **Conclusion:** The final section will summarize the key achievements of this upgrade, highlighting the expanded capabilities of the system.

2 Ontology Expansion: New Entities and Properties

To expand the functionalities offered to users by the project, I decided to introduce the concept of "Process" into the KB. A process is defined as a series of machine-operation pairs encompassing all the steps required to produce a finished product within the context of the target company for this software. For example, consider the creation of a smartphone case. The process associated with this product includes all the necessary steps involving the machines and operations defined in the KB, transforming raw materials into the final product.

This addition provides users with a powerful capability: not only can they now analyze insights and KPIs related to individual machines or operations, but they can also study data linked to entire production processes. This broader perspective is significantly more informative for analyzing the products created by the company, rather than focusing solely on specific machines or operations.

Studying individual machines or operations allows users to identify inefficiencies in specific parts of a production chain and address these issues through targeted interventions. However, analyzing entire production processes enables easy comparison of the production workflows for different products, facilitating the evaluation of more abstract concepts and better guiding users in making high-level decisions.

The new structure of the Ontology is presented in Figure 1. I will not explain aspects already covered during the course presentations; instead, this section will focus on a detailed analysis of the newly added classes, properties, and instances.

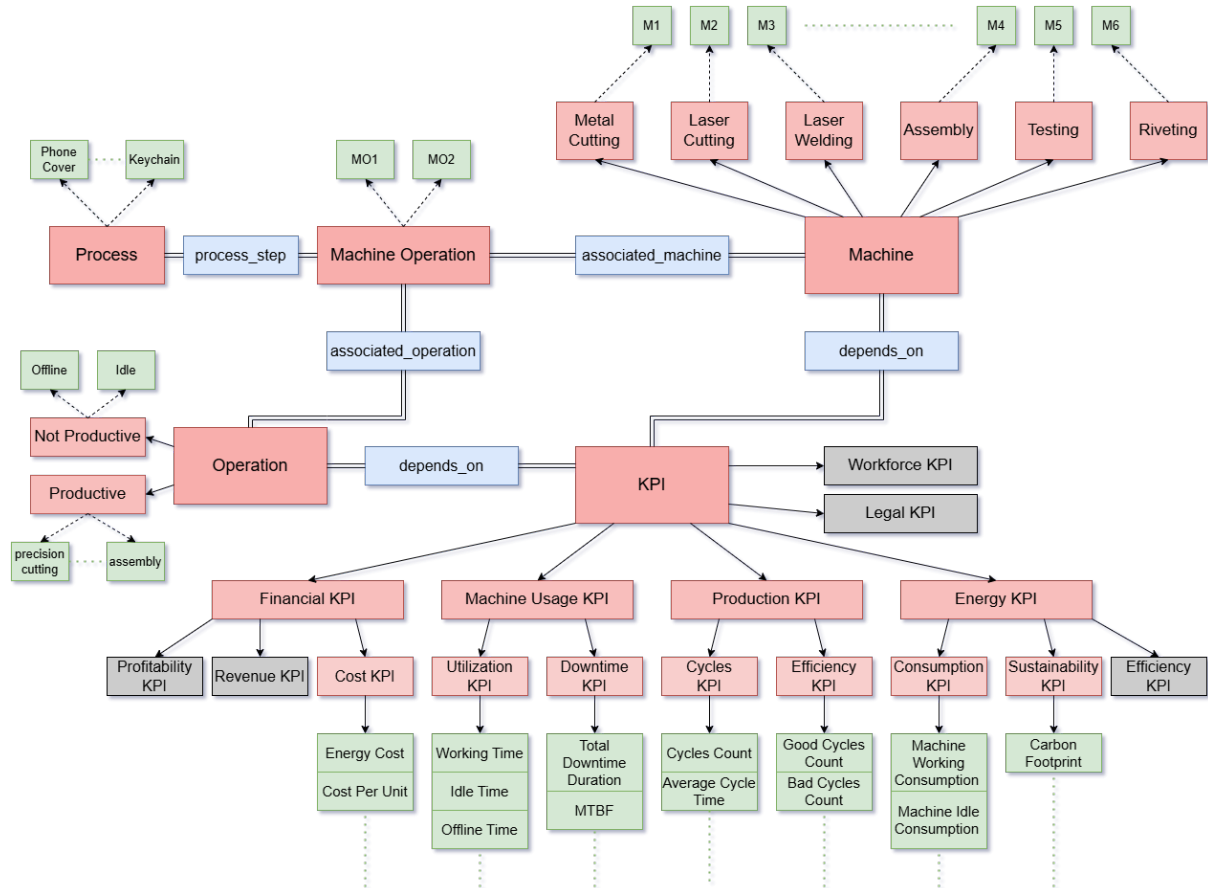


Figure 1: A schematic representation of the current content of the Knowledge Base, implemented as an ontology. The red rectangles represent classes, the green rectangles represent instances, and the blue rectangles represent object properties. The datatype properties are not explicitly shown in the image.

The first noteworthy addition is the `Process` class, which represents the previously mentioned concept. Its instances, such as `phone cover` or `keychain`, are merely illustrative examples introduced to clarify the idea. Each process is associated with a label and a description, and every instance is connected to one or more process steps through the `process_step` relationship. This structure links a process to an ordered set of steps that transform raw materials into the final product.

Each process step is an instance of the `Machine-Operation` class. These instances have their own labels and descriptions, along with an object property called `process_step_position`, which defines the ordering of the process steps. Additionally, each instance is linked to two other instances: one representing the machine and the other representing the operation, connected through the properties `associated_machine` and `associated_operation`.

Another significant modification is the revised division of operations into `Productive` and `Non-Productive`, this change was made to give more meaningful definitions to the subclasses. Previously labeled as `Generic` and `Specific`, these subclasses have been redefined: the former now represents operations that generate a product (i.e., “work” operations), while the latter encompasses operations indicating a “non-productive” machine state, such as idle or offline.

In its current state, the ontology fully fulfills its dual role within the project. First, it serves as a detailed source of information for the RAG agent, whose task is to inform the user about the product’s functionalities and guide them in executing complex queries to address their needs.

The second role is to provide support to the KPI Engine by enabling the expansion of references to groups of instances or processes into sets of machine-operation pairs, as well as supplying the formulas needed to compute the requested KPIs.

In this way, the final product, supported by the KB, becomes a highly powerful tool that enables users to receive accurate answers to natural language queries involving either standard KPIs or custom ones previously added to the KB. The product also offers the flexibility to compute a selected KPI by accumulating data over time for machines or operations, with adjustable step sizes.

3 Code Modifications: Improvements and New Implementations

To interact with the knowledge base, Topic 1 also developed and maintained a Python interface, designed to provide easy access to the ontology with methods tailored to perform specific tasks. However, the introduction of Processes rendered the interface incompatible with the new semantic structure. This was due to the need to add specific functions for creating, deleting, and utilizing processes, as well as to correct the behavior of existing functions, which no longer operated as intended.

In this section, I outline the key modifications made during this upgrade. Each of these changes is explored in detail in the following subsections:

1. **Improved Error Handling:** Replaced basic print statements with exception handling for better reliability.
2. **Optimized Instance Retrieval:** Refined the `get_closest_class_instances` function for greater precision.
3. **Custom Similarity Measure:** Implemented a tailored similarity function to enhance similarity-based search for intelligent features.
4. **Dynamic Process Management:** Added methods for creating and deleting processes.
5. **Dynamic Operation and KPI Management:** Introduced functions for adding and removing operations.

Each of these code changes was followed by a testing phase to ensure correctness. In cases where new functions were created, detailed documentation was written to make the code easily usable and maintainable.

3.1 Improved Error Handling

The initial version of the interface did not properly handle errors caused by malformed requests. In such cases, the code would simply halt execution and print a warning message. This incorrect behavior often resulted in situations that were difficult to interpret for an HTTP interface caller, as a standard error response was provided, nor was there any information regarding the error encountered.

The new implementation, paired with proper exception handling, enables a more accurate exchange of messages. All warning prints have been replaced with the raising of representative exceptions, each associated with a human-readable error message. This improvement has been applied to all pre-existing methods in the interface, as well as the newly implemented ones.

3.2 Optimized Instance Retrieval

The modification to the `get_closest_class_instances` function was driven by two distinct motivations. The method, in cases where the input label was not present in the ontology, would find the entity in the ontology whose label had the highest similarity score with the input label. However, this exhaustive search across all elements of the ontology made the function less effective, particularly in cases where the user intended to disambiguate an entity within a specific group (e.g., KPIs or Machines). In such cases, it was possible for the most similar class not to belong to the desired group.

The second motivation arose from the introduction of processes and the corresponding machine-operation instances. These additions necessitated excluding subclasses and instances of this new group, as they were not relevant as potential outputs for the function.

To address these issues, a new input parameter was added to the method, enabling the similarity search to be restricted to a specific subset of ontology entities (KPI, machine, operation or process). While unrestricted queries remain possible, they now exclude, by default, the non-relevant instances and classes of the ontology.

Additionally, a bias mechanism was introduced to slightly elevate the base similarity score of classes over instances. This adjustment reflects the fact that classes are fewer in number and aligns with the typical use case of the function, where the user is more likely seeking a match with a class rather than an instance. This prioritization ensures more accurate and contextually relevant results, especially in scenarios where class-level matching is critical.

3.3 Custom Similarity Measure

Some interface methods, such as `get_closest_class_instances` and `get_closest_kpi_formulae`, offer advanced search capabilities within the ontology. When the input label is incorrect, these methods identify the label with the highest similarity score, effectively handling issues caused by typos or imprecise inputs.

These methods have until now relied on the Levenshtein distance as the similarity measure. However, the implementation has always provided the option to modify this method using either an existing or a custom function. As part of my upgrade, I chose to create a custom similarity function tailored specifically for the case study represented by the provided dataset, in order to test the effectiveness of this approach and make this functionality even more useful.

The new "custom" similarity function is particularly beneficial when dealing with KPIs. Indeed, the KPIs in the ontology were created based on the provided data, which had the peculiar feature of supplying multiple values for each KPI, including sum, mean, maximum,

minimum, and variance. I decided to split the string comparison into the main part and the suffix, with the suffix representing one of these accumulators. By giving more weight to the main part of the label, this approach reduces interference and errors that would result from comparing improperly the suffixes, thus yielding more human-readable and accurate results. Finally, I chose that, in the absence of a suffix in the input, the "sum" value should be preferred, as it is the most significant in our case.

3.4 Dynamic Process Management

One of the key features of the implementation is its dynamic nature, enabling the KB to adapt over time to meet evolving user needs. Indeed, a key requirement was to support the dynamic creation of KPIs. With this "My Upgrade", I extended this concept beyond KPIs to encompass additional entities within the KB, enhancing its flexibility to address the user's demands, particularly for volatile and frequently changing entities like processes.

I integrated all the necessary functions into the interface to create, delete, and fully utilize these new concepts. The first two functions (creation and deletion) perform foundational operations yet fully achieve their intended purpose.

The creation function takes as input a label, a description, and a list of machine-operation pairs representing the process steps. It handles the creation of all necessary instances and relationships to properly integrate the new entity into the ontology. In contrast, the deletion function requires only the label as input and removes the process instance along with all satellite entities associated with it.

Finally, I implemented two functions, `get_steps` and `get_closest_process_steps`, to allow the KPI engine to translate references to processes received from the RAG into lists of machine-operation pairs on which to calculate the requested KPI. Given that, with the inclusion of processes, the user (and consequently the RAG) will be able to refer to these more complex constructs and not only to machines or operations explicitly. The functions also adhere to the established paradigm of providing two functions: one that offers basic functionality and another, more advanced version, that intelligently corrects possible errors in the input labels.

3.5 Dynamic Operation and KPI Management

The latest integration to the interface focuses on the dynamic management of KPIs and operations. For KPIs, the intention to enhance the creation and deletion functionalities was present even during the initial development. However, the team decided not to proceed with the implementation at the time, as the graphical interface lacked support for dynamic KPI creation due to time constraints. In this "My Upgrade," I addressed this limitation by refining the implementation and adding a "delete KPI" function to complete the feature set.

For operations, I developed functions for both their creation and deletion to enable seamless dynamic management of processes. This enhancement allows for the flexible addition of new operations, making it possible to accommodate scenarios such as the introduction of a new process in the company. For instance, when a new process involves the use of an existing machine but requires a newly defined operation, it can now be easily integrated into the system.

4 Conclusion

In this "My Upgrade," I expanded the knowledge base by incorporating the concept of processes and enhancing its dynamic capabilities. The addition of processes enables a broader representation of production workflows, facilitating analysis across entire production lines rather than just individual machines or operations. Furthermore, improvements to the Python interface have made interactions with the knowledge base more efficient, supporting dynamic management of KPIs, operations, and processes.

The recent updates, including the new error management system, improvements to existing functions, the introduction of a new similarity function, and the dynamic management of entities, have significantly enhanced the system's flexibility and reliability. These updates make the tool better suited to adapt to evolving user needs, align it with the demands of modern production environments, and more effectively support decision-making processes within the company.

The complete code for this project are available on the GitHub repository: https://github.com/Kreative-Performative-Individuals/KB/tree/giacomo_my_upgrade.